

Cálculo Diferencial

Wednesday, 14 February 2024 13:32

Lecture 4	Lecture 5	Lecture 6
- Derivatives - Gradient Descent - Multivariate Testing - Automatic Differentiation	- Fraud Detection - Bayes Theorem - Naïve Bayes - Information Theory	- Presentations - Student presentation of final project - Recommender System

Proposing a Model

- Capture the underlying relationship of observed data
 - Assume all X_i random variables (the visitors) are independent and identically distributed (iid)
 - They are mutually **independent**
 - They have **about the same probability** of occurring

Model parameter: p

p	Visitor purchases
$1-p$	Visitor does not purchase



MACHINE LEARNING UNIVERSITY

En este caso para simplificar elegimos A/B testing en el que sólo hay 2 opciones comprar o no comprar.

Maximizing Probability

- Find the probability of getting the dataset (Likelihood)
 - Each observation of the dataset is assumed to be independent
 - The (joint) probability of getting the full set of observed results is:

$$P\{\text{Data}\} = \prod_i (p)^{y_i} (1-p)^{1-y_i}$$

Handwritten notes: $y_i \in \{0, 1\}$, $p^0 = 1$, $p^1 = p$, $(1-p)^0 = 1$, $(1-p)^1 = 1-p$

Example data: $c = \{1, 0, 0, 1, \dots, 1\}$ (29), $t = \{0, 1, 0, 0, 1, 1, 0, \dots\}$ (29)

MACHINE LEARNING UNIVERSITY

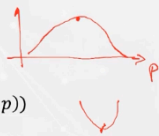
Para simplificar el cálculo de la derivada lo pasamos a log que nos convierte la operación en una suma en lugar de en una multiplicación.

Minimizing The Loss

- Minimize the Loss Function
 - For numerical and computational ease, minimize the negative log of the likelihood

$$l(p) = -\log\left(\prod_i (p)^{y_i} (1-p)^{1-y_i}\right)$$
$$l(p) = -\sum_i (y_i \log(p) + (1-y_i) \log(1-p))$$

Handwritten notes: $\log(a \cdot b) = \log(a) + \log(b)$



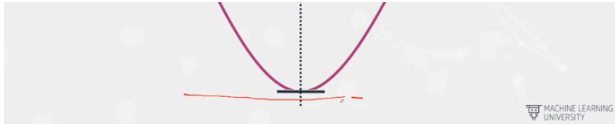
MACHINE LEARNING UNIVERSITY

Minimizing The Loss

- Graph of the negative log-likelihood function:

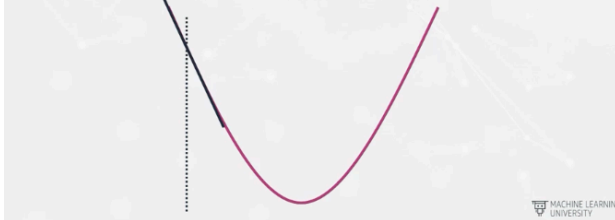


MACHINE LEARNING UNIVERSITY



Derivative of a Single Variable Function

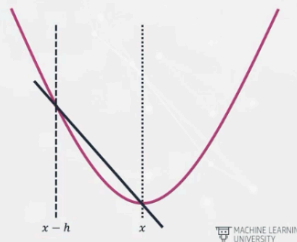
- The **derivative** is the rate of change of $f(x)$ at point x ,
 - » The **slope** of the tangent line at point x



Derivative of a Single Variable Function

- The **derivative** is defined as:

$$\lim_{h \rightarrow 0} \frac{f(x) - f(x-h)}{x - (x-h)}$$



Derivative of a Single Variable Function

- The **derivative** is defined as:

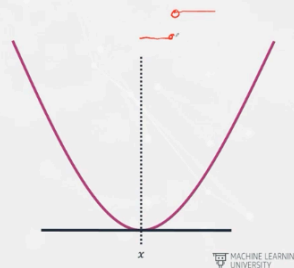
$$\lim_{h \rightarrow 0} \frac{f(x) - f(x-h)}{h}$$

» Denoted by:

$$\rightarrow \frac{df}{dx}(x)$$

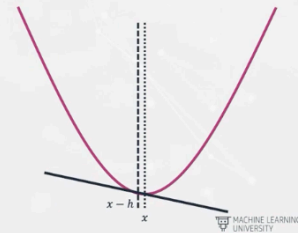
$$f'(x)$$

» assuming the limit exists



- The **derivative** is defined as:

$$\lim_{h \rightarrow 0} \frac{f(x) - f(x-h)}{h}$$



Power Rule

$$\frac{d}{dx} x^n = nx^{n-1}$$

Exponents and Logs

$$\frac{d}{dx} e^x = e^x$$

$$\frac{d}{dx} \log(x) = \frac{1}{x}$$

Trig Functions

$$\frac{d}{dx}(\sin x) = \cos x$$

$$\frac{d}{dx}(\cos x) = -\sin x$$

» There are also corollaries for *tan, csc, sec, cot*

Product Rule

(0

$$\frac{d}{dx}(g(x) \cdot h(x)) = g(x) \frac{dh}{dx}(x) + \frac{dg}{dx}(x)h(x)$$

Sigmoid Function

- Special case that is important to ML

$$\text{Sigmoid}(x) = s(x) = \frac{1}{1 + e^{-x}}$$

$$\frac{d}{dx}s(x) = s(x)(1 - s(x))$$

Minimizing the Loss (Hypothesis Testing)

- Use derivatives to find the parameter that minimizes the loss

$$l'(p) = \frac{dl}{dp} = \frac{d}{dp} \left(- \sum_y (y \log(p) + (1 - y) \log(1 - p)) \right) = 0$$

- » Find the value p , such that $l'(p) = 0$
- » This is where the loss function is 0

Derivative: Example

- How did we know this was the best way to calculate $\hat{p}_i$?

$$\hat{p}_i = \frac{X_1 + X_2 + \dots + X_{n_i}}{n_i}$$

$$\frac{\sum \#1}{\sum \text{total}}$$

- We went through a 5 step process:
 - » Multiplied all the probabilities of X_i
 - » Took the $-\log$ to separate the terms
 - » Took the derivative
 - » Set the derivative equal to 0
 - » Solved for $\hat{p}_i$

MACHINE LEARNING
UNIVERSITY

Hypothesis Testing Example

- Start with:

$$f(X_1, X_2, \dots, X_n) = p^k(1 - p)^{n-k},$$

where k = number of times $X = 1$

- » Proposed a model and multiplied all the probabilities of X .



Para simplificar el cálculo de la derivada lo pasamos a log que nos convierte la operación en una suma en lugar de en una multiplicación.

- Take the $-\log$ to separate terms:

$$\log(f(x_1, x_2, \dots, x_n)) = -k \log(p) - (n - k) \log(1 - p),$$

where k = number of times $X = 1$

- Take the derivative

$$-\log(f(x_1, x_2, \dots, x_n))' = -k \frac{1}{p} + (n - k) \frac{1}{(1 - p)},$$

where k = number of times $X = 1$

- Log Rule:

$$\frac{d}{dx} \log(x) = \frac{1}{x}$$

- Set the equation equal to zero

$$k \frac{1}{p} - (n - k) \frac{1}{(1 - p)} = 0,$$

where k = number of times $X = 1$

- Solve for $\hat{p}_i$

$$\frac{k}{p} - \frac{(n - k)}{(1 - p)} = 0$$

$$\frac{k}{p} = \frac{(n - k)}{(1 - p)}$$

where k = number of times $X = 1$

- Solve for $\hat{p}_i$

$$\frac{k}{p} = \frac{(n - k)}{(1 - p)}$$

$$(1 - p)k = p(n - k)$$

$$k - kp = np - kp$$

- Solve for $\hat{p}_i$

$$k - \cancel{kp} = np - \cancel{kp}$$

$$k = np$$

$$p = \frac{k}{n}$$

Maximum Likelihood Estimation (MLE)

- MLE is a method used to **estimate** model parameters
 - Propose a model to capture the **underlying relationship** in the dataset
 - Compute the **Likelihood** that this model generates the observed data

- » The goal is to maximize the Likelihood

$$\prod p^y (1-p)^{1-y}$$

Maximum Likelihood Estimation (MLE)

- MLE is a method used to estimate model parameters
 - » Compute the **negative log-Likelihood** (Loss function)
 - » The goal is to minimize the Loss function
 - » Find the model that **minimizes the Loss function**
 - » Maximize the Likelihood
 - » Compute the derivative of the Loss function
 - » Set derivative of the Loss function to zero
 - » Solve for the model parameters

Maximum Likelihood Estimation (MLE)

- MLE is a method used to **estimate** model parameters p
 - » Propose a model to capture the **underlying relationship** in the dataset
 - » Compute the **Likelihood** that this model generates the observed data
 - » The goal is to maximize the Likelihood

$$\prod p^y (1-p)^{1-y}$$

$[1, 0, 1, 1, 1, 1]$

Multivariate Functions

- The function being optimized currently has 1 variable, p
 - » What happens when there is another variable, $\vec{w}$?

$$l(p) = - \sum_y (y \log(p) + (1-y) \log(1-p))$$

- The function being optimized currently has 1 variable, p
 - » What happens when there is another variable, $\vec{w}$?

$$l(\vec{w}) = - \sum (y \log(f(\vec{w} \cdot \vec{x})) + (1-y) \log(1 - f(\vec{w} \cdot \vec{x})))$$

- » Is there still a derivative?

Partial Derivatives

- The **partial derivative** of f with respect to x_i is the derivative of the function
 - » Think of the other variables as **constants**

Partial Derivatives

- Example:

$$f(x_1, x_2) = x_1^3 + 5x_2^2 + x_1x_2$$

$$\frac{\partial f}{\partial x_1}(x_1, x_2) =$$

$$\frac{\partial f}{\partial x_2}(x_1, x_2) =$$

Partial Derivatives

- Example:

$$f(x_1, x_2) = x_1^3 + 5x_2^2 + x_1x_2$$

$$\frac{\partial f}{\partial x_1}(x_1, x_2) = 3x_1^2 + 0 + x_2$$

$$\frac{\partial f}{\partial x_2}(x_1, x_2) =$$

MACHINE LEARNING UNIVERSITY

Partial Derivatives

- Example:

$$f(x_1, x_2) = x_1^3 + 5x_2^2 + x_1x_2$$

$$\frac{\partial f}{\partial x_1}(x_1, x_2) = 3x_1^2 + x_2$$

$$\frac{\partial f}{\partial x_2}(x_1, x_2) = 10x_2 + x_1$$

Gradient

- The **Gradient** of f is the **vector of all partial derivatives**

- Example

$$f(x_1, x_2) = x_1^3 + 5x_2^2 + x_1x_2$$

$$\nabla f(x_1, x_2) = \begin{pmatrix} 3x_1^2 + x_2 \\ 10x_2 + x_1 \end{pmatrix}$$

MACHINE LEARNING UNIVERSITY

- The **Gradient** of f is the **vector of all partial derivatives**

- » Gradient of f (∇f) **always** points in the direction of greatest **increase** of f
- » $-\nabla f$ points in the direction of greatest **decrease**



MACHINE LEARNING UNIVERSITY

Para algunas funciones puede no ser fácil encontrar la derivada más cuando en lugar de tener 2 probabilidades como comprar-no comprar con Bernuilli tenemos hiperplanos, y aquí es donde ayuda el GRADIENT DESCENT para encontrar un método numérico en lugar de un método analítico como la derivada.

Gradient Descent

- Example plot of the negative log-likelihood by p



MACHINE LEARNING UNIVERSITY

Lo que hacemos es primero elegir un punto aleatorio en la LOSS FUNCTION y a partir de ahí moverse por los puntos en intervalos (llamado Gamma, es un parámetro, hay que elegirlo muy bien porque si es demasiado grande, los saltos no nos permitirán encontrar un punto en el que la tangente sea cercana a 0, y por tanto todas las derivadas DIVERGEN – No converge) mirando si la diferencia entre probabilidades de un punto y el siguiente es menor o igual a 0.

Gradient Descent

- Example plot of the negative log-likelihood by p

Pick a random p as a starting place

Might not be the minimum, but it's a place to start

p_1

MACHINE LEARNING UNIVERSITY

Gradient Descent

- Example plot of the negative log-likelihood by p

Move either left or right of p

Keep going until you're convinced you're at the bottom

p_1

MACHINE LEARNING UNIVERSITY

Para poder saber si el punto mínimo LOSS se encuentra a la derecha o a la izquierda (en este ejemplo se ve muy claro pero no siempre lo es) miramos si la pendiente de la derivada es positiva o negativa, entonces nos moveremos en la dirección contraria a la dirección de la derivada

Gradient Descent

- Example plot of the negative log-likelihood by p

Move in the direction **opposite** of the derivative's direction

1:05:50

MACHINE LEARNING UNIVERSITY

Ahora nos vamos moviendo en intervalos GAMMA (parámetro del modelo) hay que elegirlo muy bien porque si es demasiado grande, los saltos no nos permitirán encontrar un punto en el que la tangente sea cercana a 0, y por tanto todas las derivadas DIVERGEN – No converge) mirando si la diferencia entre probabilidades de un punto y el siguiente en menor o igual a 0.

- Example plot of the negative log-likelihood by p

Move a distance determined by the step size γ

$\gamma = 0.001$

$\gamma = 0.01$

$n = 20$

$p_5 = p_4 - \gamma f'(p_4)$

$\Delta p_5, p_1 \leq 0.01$

p_1 p_2 p_3 p_4 p_5

MACHINE LEARNING UNIVERSITY

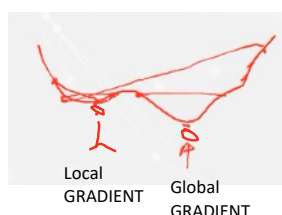
Hay que ir jugando con valor de GAMMA para optimizar el punto p , aunque cuanto más pequeño sea más tardará porque tiene que dar más saltos. EL problema de la Gamma muy grande es que corremos el riesgo de ir saltando de un lado al otro del punto sin llegar nunca a que sea menor que el umbral. Esto se llama NO CONVERGER.

Lo malo de este método numérico es que se para al encontrar un punto en el que la diferencia entre las probabilidades que está calculando sea igual o menor al parámetro elegido y puede ser que este sea un LOCAL GRADIENT y no el GLOBAL GRADIENT, que puede aparecer más adelante y por tanto tendría una p mayor, que es lo que vamos buscando (minimal LOSS)

La función AUTOGRAD de Pytorch ayuda a hacer el cálculo de GRADIENT DESCENT

PyTorch Autograd

- No need to compute derivatives by hand
- `torch.autograd`
 - PyTorch's automatic differentiation engine
 - Keeps a record of data (tensors) and all executed operations in a directed acyclic graph (DAG)
 - Tracing the graph automatically computes gradients using the chain rule



» `v.backward()` → `autograd` computes and stores the gradients in the `.grad` attribute

Hasta ahora hemos hecho una comparativa A/B pero qué pasa con el resto de opciones?

Choice Modeling (Multivariate Testing)

- Would it be better to have a row of climate pledge recommendations instead of a specific store on the website?
 - What about other sections?
 - Replace with a hero image
 - Shorten search navigation
 - Change navigation buttons
 - However sections are not independent
 - Can't do a hypothesis test on each if run together
 - What can be done?!



EL problema de estos cambios es que ya no son eventos independientes, el hecho de cambiar uno impacta seguro directamente en el resto ya que es un diseño global.

Necesitamos un modelo que nos permita medir cada una de las posibilidades como una sola

Choice Modeling (Multivariate Testing)

- Would it be better to have a row of climate pledge recommendations instead of a specific store on the website?
 - Need a model that uses each of these as a variable

$$\hat{y}_i = f(x_1, x_2, x_3, x_4)$$
 - Then do the same thing as before



Choice Modeling (Multivariate Testing)

- Put all the variations of the webpage out into production and collection **all** the data

Visitor Tag	Purchased product	Amazon Choice	Hero image	Short search buttons	New
84920394	0	0	1	1	0
84223934	1	0	0	0	1
85202368	0	0	1	0	0
84342229	1	1	1	1	1
83393332	0	1	1	1	0
84352232	0	0	0	0	1
84432302	1	1	1	1	1
84333736	0	1	0	0	1
84554332	0	0	0	1	0
83354443	1	1	1	0	1
84430232	0	1	0	0	0
48886763	1	0	0	1	0
83356679	1	0	1	0	1
84090932	0	1	0	1	0



Vamos a seguir buscando el Maximo valor de P MLE pero en lugar de para una p y p-1 para cada una de las X

Maximum Likelihood Estimation (MLE)

- Used to estimate model parameters
 - Propose a model to capture the underlying relationship in the dataset x_1, x_2, x_3, x_4
 - Compute the Likelihood that the model generates the observed data y_i
 - The goal is to maximize the Likelihood
 - Compute the negative log-Likelihood (Loss function)
 - The goal is to minimize the Loss function

El modelo que se propone para esta situación es la regresión logística, donde todas nuestras features X s tendrán cada una un peso W y la posible salida será solo con 2 posibles valores. Ahora el cálculo de nuestra P se complica porque ya no es p y $p-1$ ahora es una ecuación lineal a la que se le puede aplicar una regresión.

Logistic Regression

- Logistic Regression models the probability that an observation x is in the positive class ($y = 1$):

$$P(y = 1) = \frac{1}{1 + e^{-\vec{w} \cdot \vec{x}}} = f(\vec{w} \cdot \vec{x})$$

- And because there are only two possible outcomes, 1 and 0

$$P(y = 0) = 1 - f(\vec{w} \cdot \vec{x})$$

Ahora es un sigmoide

Multivariate Testing Example

- Process review:
 - Multiplied all the probabilities of X_i $p^k (1-p)^{n-k}$

$$f(x_1, x_2, \dots, x_n) = \left(\frac{1}{1 + e^{-\vec{w} \cdot \vec{x}}} \right)^k \left(1 - \frac{1}{1 + e^{-\vec{w} \cdot \vec{x}}} \right)^{n-k},$$

where $k = \text{number of times } x = 1$

Multivariate Testing Example

- Process review:
 - Took the -log to separate the terms

$$-\log(f(x_1, x_2, \dots, x_n)) = -k \log\left(\frac{1}{1 + e^{-\vec{w} \cdot \vec{x}}}\right) - (n - k) \log\left(1 - \frac{1}{1 + e^{-\vec{w} \cdot \vec{x}}}\right),$$

where $k = \text{number of times } x = 1$

Multivariate Testing Example

- Process review:
 - Took the derivative

$$-\log(f(x_1, x_2, \dots, x_n))' = -k \frac{1}{1 + e^{-\vec{w} \cdot \vec{x}}} - (n - k) \frac{-1}{\left(1 - \frac{1}{1 + e^{-\vec{w} \cdot \vec{x}}}\right)}$$

$$-\log(f(x_1, x_2, \dots, x_n))' = -k(1 + e^{-\vec{w} \cdot \vec{x}}) + (n - k) \frac{1}{\left(1 - \frac{1}{1 + e^{-\vec{w} \cdot \vec{x}}}\right)}$$

where $k = \text{number of times } x = 1$

- Process review:
 - Took the derivative :

$$-\log(f(x_1, x_2, \dots, x_n))' = -k(1 + e^{-\vec{w} \cdot \vec{x}}) + (n - k) \frac{1}{(1 + e^{-\vec{w} \cdot \vec{x}})}$$

- » This is getting messy
- » There's an easier way to finish this process that works for almost all situations

Aquí de nuevo el método analítico será difícil que funcione y si lo hace será largo, así que podemos de nuevo hacer uso de un método numérico

Logistic Regression

- Logistic Regression models the probability that an observation x is in the positive class ($y = 1$):

Sigm

$$P(y = 1) = \frac{1}{1 + e^{-\vec{w} \cdot \vec{x}}} = f(\vec{w} \cdot \vec{x})$$

Gradient descent finds these weights

- And because there are only two possible outcomes, 1 and 0

Classification

$$P(y = 0) = 1 - f(\vec{w} \cdot \vec{x})$$

Este método de regresión logística toma el nombre de que WX es una regresión lineal en lugar de Bernuilli, a la cual si le aplicamos el sigmoide lo transforma en una probabilidad como resultado $P(y=1)$. La función de sigmoide tendrá ahora esta pinta, y estamos buscando el punto medio que minimiza el LOSS function, ya que si es mayor que ese punto será clase 1 y si es menor será clase 0.

Lecture 4 Review

- Maximum Likelihood Estimation (MLE) is a method used to estimate model parameters
- 1. » Propose a model (with its parameters) to capture the underlying relationship in the dataset
- 2. » Compute the Likelihood (joint probability) that this model generates the observed data - the goal is to maximize the Likelihood

Ahora el modelo en lugar de ser Bernuilli será una Regresión Logística.